MINI PROJECT REPORT

CAMPUS ROUTE FINDER USING GRAPHS

Submitted By: Deepanshu Mittal
Roll Number: 2556446
Course: BTech CSE A
Subject: Data Structures and Algorithms

---

1. INTRODUCTION

The Campus Route Finder is a mini project developed using the concepts of Graphs in Data Structures and Algorithms. It represents different locations on a campus and the paths connecting them.

The project helps users find the shortest route between two locations using Breadth-First Search (BFS). It also uses Depth-First Search (DFS) to identify reachable and disconnected locations.

The project is implemented in the C programming language using a menu-driven interface.

2. OBJECTIVES

The main objectives of this project are:

- To understand the concept of graphs.
- To represent campus locations and connections.
- To find the shortest path between two locations using BFS.
- To identify reachable locations using DFS.
- To display disconnected locations.
- To implement graph traversal algorithms in C.

3. PROBLEM STATEMENT

Finding routes between different locations on a campus can be difficult when several paths are available. This project provides a simple way to represent campus locations as a graph and determine routes between them.

The user selects a source and destination, and the program finds the shortest path in terms of the number of edges when a route exists.

4. SOFTWARE AND HARDWARE REQUIREMENTS

Software Requirements

- Programming Language: C
- Compiler: GCC, Code::Blocks, or another C compiler
- Operating System: Windows, Linux, or macOS

Hardware Requirements

- Computer or laptop
- Basic memory and storage
- Keyboard and display

5. DATA STRUCTURE USED

A graph consists of vertices and edges.

- **Vertices:** Represent campus locations.
- **Edges:** Represent direct paths between locations.

The program contains eight locations:

1. Main Gate
2. Library
3. Block 1
4. Block 2
5. Cafeteria
6. Auditorium
7. Parking
8. Lab

The current implementation uses an **adjacency matrix** to represent connections. A value of 1 indicates a direct connection between two locations, while 0 indicates no direct connection.

The graph is undirected because each connection works in both directions.

6. ALGORITHMS USED

6.1 Breadth-First Search (BFS)

BFS explores the graph level by level using a queue. It visits neighbouring locations before moving to locations farther away.

Steps:

1. Mark the source location as visited.
2. Add the source to the queue.
3. Remove a location from the queue.
4. Visit its unvisited neighbours and record their parents.
5. Continue until the destination is reached or the queue becomes empty.
6. Reconstruct and display the shortest path using the parent information.

BFS finds a shortest path in an unweighted graph in terms of the number of edges.

6.2 Depth-First Search (DFS)

DFS explores a path as far as possible before returning to explore another path. This implementation uses recursion.

Steps:

1. Start from the selected location.
2. Mark the current location as visited.
3. Visit each unvisited neighbouring location recursively.
4. After traversal, display all visited locations.
5. Display locations that were not visited as disconnected from the selected starting location.

7. SYSTEM IMPLEMENTATION

The program is implemented in C and provides the following menu options:

1. Display Campus Locations
2. Find Shortest Route (BFS)
3. Find Reachable Locations (DFS)
4. Exit

The user selects an option from the menu. For route finding, the user enters the source and destination location numbers. The program validates the location numbers and displays the route or an appropriate message.

8. TEST CASES

The following test cases can be used to verify the program.

Test Case 1: Display Campus Locations

Input: Select option 1.

Result: All eight campus locations should be displayed.

Test Case 2: Find Shortest Route

Input: Select option 2, source = 0, destination = 5.

Result: The shortest route should be displayed as a sequence of connected locations.

Test Case 3: Find Reachable Locations

Input: Select option 3, starting location = 0.

Result: The program should display all locations reachable from the Main Gate and list any locations that are disconnected from it.

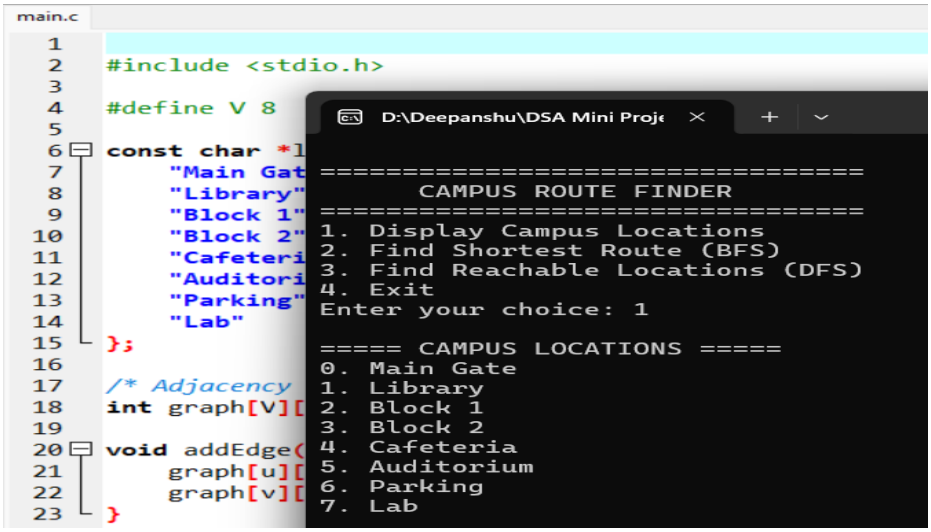
Test Case 4: Invalid Location

Input: Enter a location number outside the range 0–7.

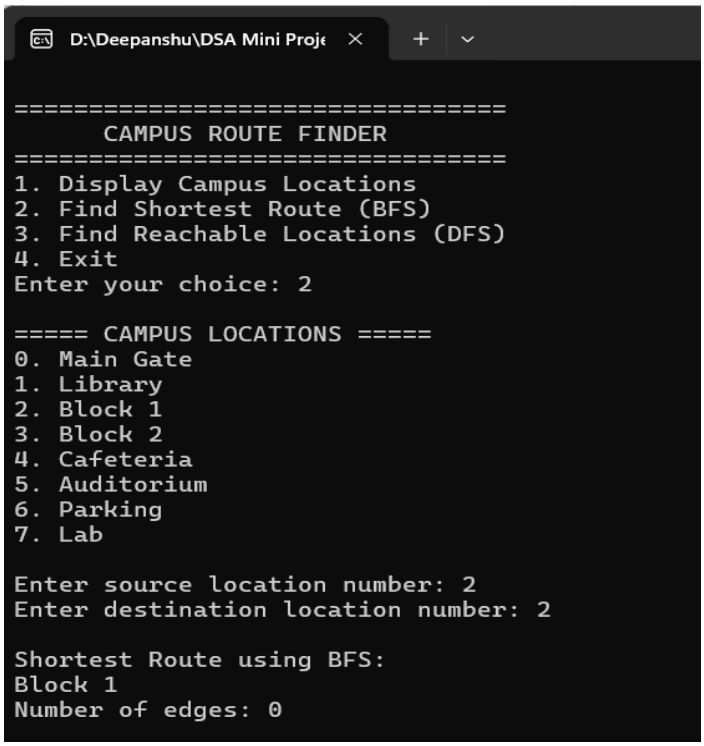
Result: The program should display an invalid location message.

9. OUTPUT SCREENSHOTS

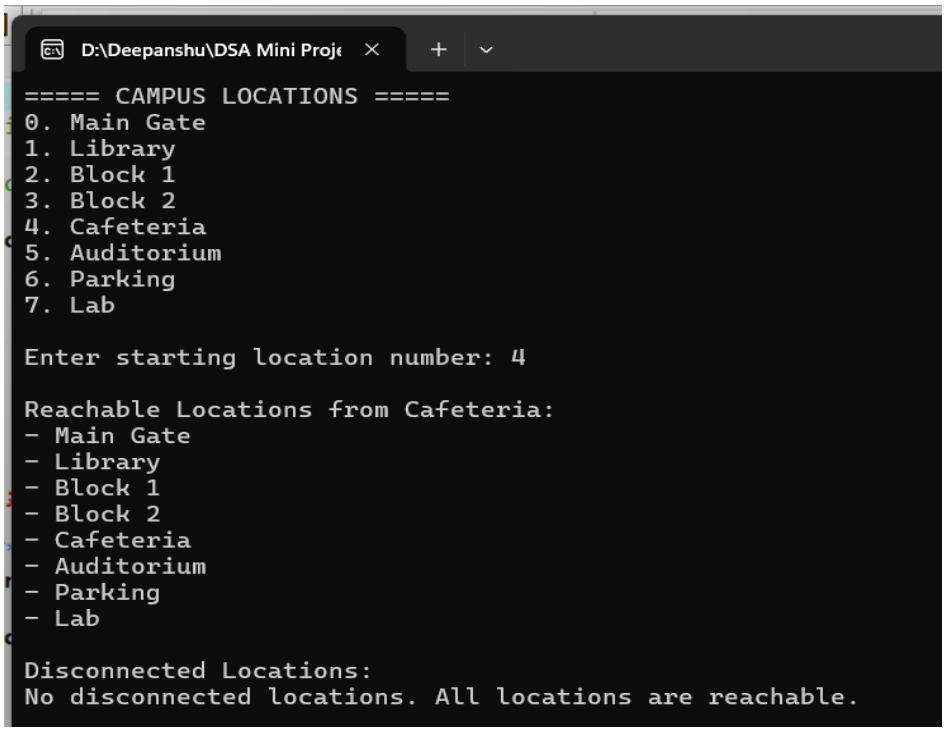
Screenshot 1: Main Menu and Campus Locations



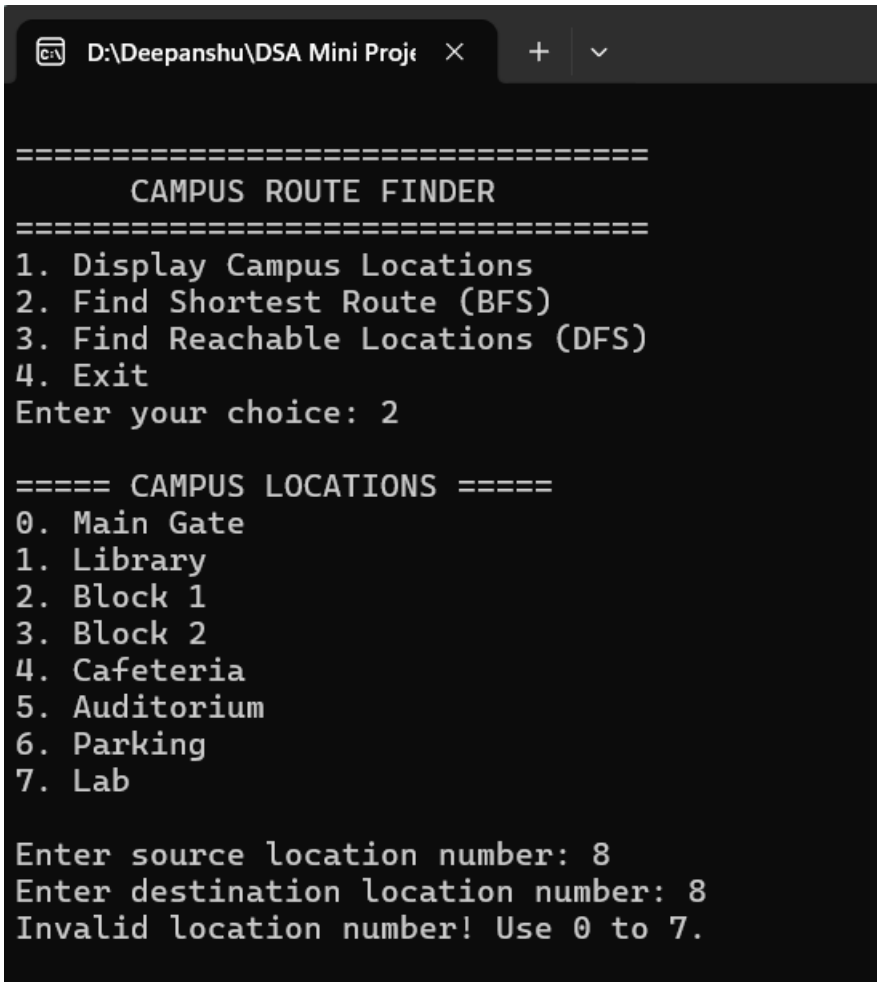
Screenshot 2: Shortest Route Using BFS



Screenshot 3: Reachable and Disconnected Locations Using DFS



Screenshot 4: Invalid Location Input



10. TIME AND SPACE COMPLEXITY

Let **V** represent the number of vertices and **E** represent the number of edges.

The current implementation uses an adjacency matrix.

- **BFS Time Complexity:** $O(V^2)$
- **DFS Time Complexity:** $O(V^2)$
- **Graph Storage Complexity:** $O(V^2)$

The traversal time is $O(V^2)$ because the program scans a complete row of the adjacency matrix for each visited vertex.

11. ADVANTAGES

- Simple and easy-to-use menu-driven interface.
- Demonstrates graph representation and traversal.
- Finds shortest routes in an unweighted graph.
- Identifies reachable and disconnected locations.

- Validates location numbers before processing.

12. LIMITATIONS

- The campus locations and connections are predefined.
- The program does not calculate real-world distances or travel times.
- All edges are treated equally.
- The current implementation uses an adjacency matrix rather than an adjacency list.

13. FUTURE SCOPE

The project can be improved by:

- Using an adjacency list for graph representation.
- Allowing users to add or remove locations and routes.
- Assigning distances or weights to edges.
- Finding minimum-distance routes using Dijkstra's algorithm.
- Developing a graphical interface or campus map.

14. CONCLUSION

The Campus Route Finder demonstrates how graphs can be used to represent campus locations and their connections. BFS is used to find the shortest path in an unweighted graph, while DFS is used to explore reachable locations.

This project provides practical experience with graph representation, traversal algorithms, queues, recursion, and menu-driven programming in C.

15. REFERENCES

1. Data Structures and Algorithms course notes.
2. C Programming Language reference materials.
3. Graph traversal concepts: Breadth-First Search and Depth-First Search.